

MODULE 2

Deep Learning & Activations (Thèse)

Doctorant	Tidiane DIALLO
Directeur	Pr. Abdoul Aziz Ciss — EPT
Période	Semaines 3-5 Avril-Mai 2026
Niveau	Bases → Intermédiaire
Durée estimée	8-10 heures

Chapitre 1 — Le Perceptron

Le perceptron est l'unité de base d'un réseau de neurones. Il imite le neurone biologique: reçoit des signaux, les combine, et décide de s'activer ou non.

$$\blacksquare = \text{activation}(W \cdot x + b)$$

Composant	Rôle	Valeur typique
x (entrées)	Signal reçu	Vecteur réel
W (poids)	Importance de chaque entrée	Initialisé aléatoirement
b (biais)	Décalage du seuil d'activation	Initialisé à 0
activation()	Introduit la non-linéarité	ReLU, GELU, Sigmoid...
$\blacksquare$ (sortie)	Signal émis vers la couche suivante	Scalaire ou vecteur

Chapitre 2 — Réseau Multicouche (MLP)

Un MLP (Multi-Layer Perceptron) empile plusieurs couches de neurones. Les couches intermédiaires s'appellent "couches cachées". La profondeur (nombre de couches) donne son nom au "Deep Learning".

```
import torch.nn as nn

# Architecture MLP: 784 → 256 → 128 → 10
model = nn.Sequential(
    nn.Linear(784, 256), # couche 1
    nn.ReLU(), # activation
    nn.Linear(256, 128), # couche 2
    nn.GELU(), # activation GELU (thèse!)
    nn.Linear(128, 10), # couche de sortie
)

# Forward pass: calculer la sortie
import torch
x = torch.randn(1, 784) # 1 image de 784 pixels
output = model(x)
print(output.shape) # torch.Size([1, 10])
```

Chapitre 3 — Fonctions d'Activation (CŒUR DE LA THÈSE)

Pourquoi ce chapitre est central pour ta thèse

Ta thèse porte sur les ATTAQUES D'EXTRACTION pour les réseaux au-delà de ReLU. Les méthodes existantes exploitent la structure particulière de ReLU (point de coude). Les activations modernes (GELU, SiLU) n'ont pas ce point de coude, rendant les attaques connues inopérantes. C'est le verrou scientifique que tu dois résoudre.

Activation	Formule	Propriété clé	Utilisée dans
ReLU	$\max(0, x)$	Point de coude en 0 → EXPLOITABLE par attaque	Réseaux classiques
GELU	$x \cdot \Phi(x)$	Lisse, sans discontinuité → RÉSISTANTE?	GPT, BERT, LLMs
SiLU/Swish	$x \cdot \sigma(x)$	Non-monotone, lisse → RÉSISTANTE?	EfficientNet, vision
Sigmoid	$1/(1+e^{\blacksquare})$	Sortie entre [0,1]	Couche de sortie
Tanh	$(e^{\blacksquare}-e^{\blacksquare})/(e^{\blacksquare}+e^{\blacksquare})$	Sortie entre [-1,1]	RNN, LSTM

Propriété critique de ReLU pour les attaques

ReLU est défini par deux régimes linéaires: $f(x)=0$ pour $x \leq 0$ et $f(x)=x$ pour $x > 0$. La frontière entre ces deux régimes (le "kink" ou point de coude) est exploitable par les attaques cryptanalytiques de Carlini et al. (2020). En envoyant des requêtes ciblées, l'attaquant peut localiser exactement ce point de coude et ainsi déduire les paramètres du réseau.

Challenge pour GELU et SiLU

GELU = $x \cdot \Phi(x)$ où Φ est la fonction de répartition de la loi normale. Cette fonction est infiniment dérivable (C^∞), sans aucun point de coude. Il n'y a pas de frontière binaire exploitable. Ta thèse doit trouver une nouvelle propriété mathématique exploitable, ou prouver que l'extraction est impossible/exponentiellement coûteuse.

Chapitre 4 — Entraînement d'un DNN

Étape	Opération	Code PyTorch
1. Forward pass	Calculer la sortie $\hat{y} = f(x)$	<code>output = model(x)</code>
2. Calcul de la perte	$L = \text{loss}(\hat{y}, y)$	<code>loss = criterion(output, y)</code>
3. Backpropagation	Calculer ∇L par rapport à tous les poids	<code>loss.backward()</code>
4. Mise à jour	$w \leftarrow w - \alpha \cdot \nabla L$	<code>optimizer.step()</code>
5. Reset gradient	Remettre les gradients à 0	<code>optimizer.zero_grad()</code>

```
# Boucle d'entraînement complète (template standard)
model = MonReseau()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    for x_batch, y_batch in dataloader:
        optimizer.zero_grad() # étape 5 (avant)
        output = model(x_batch) # étape 1
        loss = criterion(output, y_batch) # étape 2
        loss.backward() # étape 3
        optimizer.step() # étape 4
```